

POKEMON TOURNAMENT

Manuale Completo del Codice
Guida dettagliata per comprendere ogni componente del gioco

Linguaggio	Python 3.12+
Framework grafico	Pygame 2.5
Architettura	Multi-thread con code asincrona
File principali	main.py, finestra_tk.py, battaglia.py, torneo.py, ai.py, danno.py, pokemon.py, dati.py
Pokemon disponibili	809 (da Pokedex JSON)

Indice dei Contenuti

1.	Panoramica del Gioco	Come funziona il torneo Pokémon
2.	Struttura dei File	Organizzazione del progetto
3.	Costanti e Configurazione	Dimensioni, colori, layout
4.	Sistema di Comunicazione	Code tra logica e grafica
5.	main.py — Punto d'ingresso	Avvio dei thread
6.	dati.py — Caricamento Dati	Lettura del Pokedex JSON
7.	pokemon.py — Stato di Battaglia	Oggetto Pokemon in battaglia
8.	danno.py — Calcolo Danni	Formule di danno e cura
9.	ai.py — Intelligenza Artificiale	Logica CPU facile/media/difficile
10.	torneo.py — Gestione Torneo	Bracket a eliminazione diretta
11.	battaglia.py — Motore di Battaglia	Logica del combattimento
12.	finestraTk.py — Grafica Pygame	Rendering, animazioni, eventi
13.	Sistema Temi Giorno/Notte	Colori e immagini per tema
14.	Sistema di Animazioni	Particelle, shake, scia, onde
15.	Schermate del Gioco	Difficoltà, selezione, tabellone, battaglia
16.	Aggiungere Contenuti	Nuovi Pokémon, wallpaper, font

1. Panoramica del Gioco

Pokemon Tournament è un gioco a turni per un giocatore contro la CPU. Il giocatore sceglie un Pokemon e partecipa a un torneo a eliminazione diretta con altri 15 Pokemon (selezionati in base alla difficoltà). Ogni scontro è un combattimento 1vs1 con mosse simultanee.

Flusso di gioco

Fase	Descrizione
Avvio	main.py lancia due thread: logica (battaglia.py/torneo.py) e grafica (finestraTk.py via Pygame)
Difficoltà	Il giocatore sceglie Facile / Media / Difficile — determina il pool di avversari
Selezione	Sceglie il proprio Pokemon da una griglia di 809 disponibili
Tabellone	Il torneo viene generato con 16 Pokemon (il giocatore + 15 CPU), bracket casuale
Battaglia	Ad ogni round il giocatore sceglie una mossa; la CPU sceglie in base alla difficoltà
Round	Le mosse vengono eseguite simultaneamente; chi arriva a 0 HP perde
Avanzamento	Dopo ogni battaglia compare il tabellone aggiornato, poi si passa al round successivo
Campione	Se il giocatore vince tutti e 4 i round (ottavi, quarti, semifinale, finale) vince il torneo

Mosse disponibili in battaglia

Mossa	Effetto
Attacco	Usa la statistica ATK del Pokemon. Danno fisico diretto.
Attacco Speciale	Usa la statistica SpA. Può avere moltiplicatore di tipo (x0, x0.5, x1, x2).
Pozione Normale	Ripristina il 100% degli HP. L'eccesso va in DEF/SpD. Limitata per round.
Pozione Speciale	Come la pozione normale ma conta separatamente. Limitata per round.

Le mosse sono **SIMULTANEE**: entrambi i combattenti scelgono nello stesso turno. Se uno attacca e l'altro si cura, la cura avviene prima. In caso di doppio KO, perde il Pokemon più lento.

2. Struttura dei File

Il progetto è suddiviso in file Python con responsabilità distinte, più una cartella dati con immagini e JSON.

File / Cartella	Ruolo
main.py	Punto d'ingresso. Crea i thread logica e grafica, li collega tramite due Queue.
finestraTk.py	Tutta la grafica (Pygame). Gestisce eventi mouse, disegno, animazioni, temi. ~1350 righe.
battaglia.py	Motore di combattimento. Gestisce turni, mosse simultanee, pozioni, KO.
torneo.py	Genera e gestisce il bracket a eliminazione. Avanza round, determina il campione.
ai.py	Logica decisionale della CPU in base alla difficoltà (facile/media/difficile).
danno.py	Calcola i danni fisici e speciali, cura, moltiplicatori di tipo.
pokemon.py	Classe che rappresenta lo stato di un Pokemon durante la battaglia (HP, difesa attuale...).
dati.py	Carica pokedex.json e types.json. Fornisce la lista di tutti i Pokemon.
PokemonGame/pokedex.json	Database con statistiche e tipi di 809 Pokemon.
PokemonGame/types.json	Matrice di efficacia dei tipi (es. Fuoco vs Acqua = 0.5x).
PokemonGame/pokemon_images/	Immagini PNG dei Pokemon (nome.png o nome-forma.png).
PokemonGame/style/	Immagini di stile: pannelli, sole, luna, nuvole, font TTF.
PokemonGame/wallpaper_dark/	Sfondi per la battaglia in modalità notte.
PokemonGame/wallpaper_light/	Sfondi per la battaglia in modalità giorno.

3. Costanti e Configurazione

Tutte le costanti di layout e configurazione sono definite all'inizio di `finestra_tk.py` come variabili modulo. Cambiarle qui modifica l'intero gioco.

Finestra e timing

Costante	Valore	Significato
W, H	1280, 760	Dimensioni della finestra in pixel
BAR	44	Altezza della barra superiore in pixel
TICK	50	Target FPS (frame al secondo = $1000/TICK = 20$ FPS)

Sprite e posizioni di battaglia

Costante	Valore	Significato
SPR_SEL	36	Raggio del cerchio sprite nella griglia di selezione
SPR_PAN	72	Raggio del cerchio sprite nel pannello statistiche
SPR_B	320	Dimensione sprite in battaglia (320×320 px)
SPR_OFF	0.05	Offset verticale dello sprite rispetto al centro
SPR_CER_OFF	0.25	Offset verticale dello sprite dentro il cerchio
GX	50	X del giocatore in battaglia (angolo in alto a sinistra dello sprite)
GY	80	Y del giocatore in battaglia
AX	W-370	X dell'avversario in battaglia
AY	140	Y dell'avversario in battaglia

Griglia di selezione Pokemon

Costante	Valore	Significato
GRIGLIA_CELLA_W	200	Larghezza di ogni cella nella griglia
GRIGLIA_CELLA_H	96	Altezza di ogni cella nella griglia
GRIGLIA_GAP	6	Spazio tra celle (orizzontale e verticale)
GRIGLIA_ORIG_X	238	X dell'angolo in alto a sinistra della griglia
GRIGLIA_ORIG_Y	54	Y dell'angolo in alto a sinistra della griglia (BAR+10)
GRIGLIA_COLONNE	5	Numero di colonne nella griglia
PANNELLO_L_W	230	Larghezza del pannello statistiche a sinistra

Log e zona battaglia bassa

Costante	Valore	Significato
LOW	282	Altezza della zona inferiore (log + pulsanti mossa) in pixel
LOG_N	14	Numero di righe di log visibili simultaneamente

Tabellone torneo

Costante	Valore	Significato
BRACKET_TOP	BAR+35	Y di inizio del tabellone
BRACKET_H	H-BAR-65	Altezza totale del tabellone
BRACKET_BOX_W	155	Larghezza di ogni box match
BRACKET_BOX_H	56	Altezza di ogni box match
BRACKET_GAP	18	Spazio tra le colonne del bracket

Le coordinate X di ogni colonna del tabellone (R16_L_X1, QF_L_X1, SF_L_X1, FINAL_X1 e i loro simmetrici a destra) vengono calcolate automaticamente a partire dal centro dello schermo e dal BRACKET_GAP, in modo da essere sempre bilanciati.

4. Sistema di Comunicazione tra Thread

Il gioco usa due thread separati: uno per la logica (battaglia, AI, torneo) e uno per la grafica (Pygame). Comunicano tramite due oggetti queue.Queue thread-safe.

Le due code

Coda	Direzione	Uso
coda_comandi	Logica → Grafica	La logica invia messaggi alla GUI (es: disegna la battaglia, mostra animazione)
coda_risposte	Grafica → Logica	La GUI invia input dell'utente (es: ha cliccato Attacco, ha scelto Pikachu)

Messaggi dalla logica alla grafica (coda_comandi)

Tipo messaggio	Descrizione
difficolta	Mostra schermata scelta difficoltà. Passa il pool di Pokemon decorativi.
selezione	Mostra griglia selezione. Passa pool Pokemon disponibili e difficoltà.
tabellone	Mostra bracket torneo. Passa i dati del bracket e messaggio eventuale.
battaglia_inizia	Avvia una battaglia. Passa i dati di giocatore e avversario.
aggiorna	Aggiorna HP/pozioni visualizzati durante la battaglia.
log	Aggiunge una riga al log di battaglia con testo e colore.
chiedi_mossa	Attiva i pulsanti mossa (è il turno del giocatore).
anim_attacco	Lancia animazione di attacco fisico (scatto, onde, particelle).
anim_attacco_doppio	Attacco simultaneo di entrambi.
anim_speciale	Lancia animazione attacco speciale (scia colorata). Passa moltiplicatore.
anim_cura	Lancia animazione pozione (bolle colorate).
anim_ko	Lancia animazione KO (dissolvenza).
risultato	Fine battaglia: mostra messaggio e pulsante Continua.
campione	Fine torneo: mostra schermata campione.

Messaggi dalla grafica alla logica (coda_risposte)

Tipo messaggio	Descrizione
difficolta	L'utente ha scelto la difficoltà. Valore: 'facile'/'media'/'difficile'.
pokemon	L'utente ha scelto il suo Pokemon. Valore: dizionario con dati Pokemon.
mossa	L'utente ha scelto una mossa. Valore: 'attacco'/'attacco_speciale'/'pozione_normale'/'pozione_speciale'.

continua	L'utente ha cliccato Continua (dopo risultato o tabellone).
indietro	L'utente ha cliccato Indietro nella selezione Pokemon.
esci	L'utente ha chiuso la finestra.

5. main.py — Punto d'ingresso

main.py è il file da eseguire per avviare il gioco. Ha il solo compito di creare i due oggetti principali, collegarli tramite le code e avviare i thread.

Struttura di main.py

```
dati_gioco = DatiGioco('PokemonGame/pokedex.json', 'PokemonGame/types.json')
# Carica tutti i 809 Pokemon e la matrice dei tipi dal disco

finestra = Finestra('PokemonGame/pokemon_images', 'PokemonGame')
# Crea l'oggetto grafico Pygame con percorsi alle immagini

logica = LogicaGioco(dati_gioco, finestra.coda_comandi, finestra.coda_risposte)
# Crea la logica collegandola alle due code della finestra

thread_logica = threading.Thread(target=logica.avvia, daemon=True)
# Il thread logica è daemon: si ferma quando si chiude la finestra

finestra.avvia(thread_logica)
# Avvia il loop Pygame nel thread principale (richiesto da macOS/Windows)
```

PERCHÉ DUE THREAD? Pygame richiede di girare nel thread principale. La logica di gioco (AI, calcoli, attese) deve girare in un thread separato per non bloccare il rendering. Le Queue garantiscono che i due thread comunichino in modo sicuro senza race condition.

6. dati.py — Caricamento Dati

dati.py carica al momento dell'avvio il database dei Pokemon e la tabella dei tipi. Entrambi vengono letti da file JSON.

Struttura del pokedex.json

Ogni Pokemon è un dizionario con questi campi:

Campo	Tipo	Descrizione
nome	string	Nome del Pokemon (es: 'Pikachu'). Usato anche come nome file immagine.
tipi	list	Lista di 1 o 2 tipi (es: ['Electric'] o ['Water','Flying']).
stats	dict	Dizionario con: hp, attack, defense, sp_attack, sp_defense, speed.

Struttura del types.json

Matrice NxN (18 tipi × 18 tipi) che contiene il moltiplicatore di danno quando un attacco di tipo X colpisce un Pokemon di tipo Y. Valori possibili: 0 (immune), 0.5 (non molto efficace), 1 (normale), 2 (super efficace).

Esempio: `types['Fire']['Grass'] = 2.0` → il fuoco fa il doppio dei danni all'Erba. Il calcolo finale per un Pokemon dual-type moltiplica entrambi i valori (es: Volante+Ghiaccio vs Fuoco: $1.0 \times 0.5 = 0.5x$).

7. pokemon.py — Stato di Battaglia

La classe `PokemonBattaglia` rappresenta un Pokemon durante uno scontro. Mantiene i valori attuali (HP, difesa) che cambiano nel corso della battaglia, separati dalle statistiche base del Pokedex.

Attributi principali

Attributo	Descrizione
<code>nome</code>	Nome del Pokemon
<code>tipi</code>	Lista dei tipi (copiata dal Pokedex)
<code>stats</code>	Dizionario statistiche base (invariato durante la battaglia)
<code>hp_attuale</code>	HP correnti. Inizia uguale a <code>stats['hp']</code> . Scende quando si riceve danno.
<code>difesa_attuale</code>	Difesa corrente. Può aumentare con la pozione (eccesso HP → DEF).
<code>sp_difesa_attuale</code>	Difesa speciale corrente. Funziona come <code>difesa_attuale</code> .
<code>pozioni_normali</code>	Numero di pozioni normali rimaste per questo round.
<code>pozioni_speciali</code>	Numero di pozioni speciali rimaste per questo round.

Perché separare `hp_attuale` da `stats['hp']`?

`stats['hp']` è il valore massimo base, mai modificato. `hp_attuale` è il valore attuale che scende in battaglia. Questo permette di calcolare percentuali per le barre HP e di sapere quando un Pokemon è KO (`hp_attuale <= 0`).

8. danno.py — Calcolo Danni e Cura

danno.py contiene le formule per calcolare quanti HP vengono tolti o ripristinati in ogni azione.

Attacco fisico

```
danno = max(1, attaccante.stats['attack'] - difensore.difesa_attuale)
# Il danno minimo è sempre 1 (non può essere 0 o negativo)
# difesa_attuale può crescere con le pozioni
```

Attacco speciale

```
moltiplicatore = calcola_tipo(tipo_attaccante, tipi_difensore, types_data)
# Calcola x0, x0.5, x1 o x2 in base alla tabella dei tipi

danno = max(0, int(attaccante.stats['sp_attack'] * moltiplicatore
- difensore.sp_difesa_attuale))
# Se il moltiplicatore è 0 (immune), il danno è esattamente 0
```

Pozione

```
cura = difensore.stats['hp'] # Ripristina il 100% degli HP base
hp_prima = difensore.hp_attuale
difensore.hp_attuale = difensore.stats['hp']
eccesso = cura - (difensore.stats['hp'] - hp_prima)

if eccesso > 0:
    # L'HP era già alto: l'eccesso va a potenziare la difesa
    difensore.difesa_attuale += eccesso // 2
    difensore.sp_difesa_attuale += eccesso // 2
```

Il meccanismo dell'eccesso rende le pozioni usate in anticipo più efficaci: curarsi quando si è quasi integri aumenta DEF e SpD, creando una scelta strategica (curare subito o aspettare di perdere HP?).

9. ai.py — Intelligenza Artificiale

La CPU sceglie la mossa in base alla difficoltà selezionata dal giocatore. Le tre difficoltà usano algoritmi sempre più sofisticati.

Difficoltà	Comportamento CPU
Facile	Sceglie una mossa a caso tra quelle disponibili. Non usa strategia. Il pool di avversari è composto dai 15 Pokemon più deboli (basso HP/ATK totale).
Media	Valuta le mosse: preferisce l'attacco speciale se il moltiplicatore è favorevole, usa la pozione se gli HP sono sotto il 40%. Pool di 15 Pokemon casuali.
Difficile	Simula entrambe le mosse possibili e sceglie quella con il miglior risultato atteso. Tiene conto dei danni ricevuti, dei danni inflitti e dello stato delle pozioni. Pool dei 15 Pokemon più forti.

Selezione del pool avversari

```
# Calcola un 'punteggio' per ogni Pokemon
punteggio = stats['hp'] + stats['attack'] + stats['sp_attack'] + stats['speed']

# Facile: prende i 15 con punteggio più basso
pool_facile = sorted(tutti, key=punteggio)[:15]

# Media: 15 Pokemon casuali dall'intero Pokedex
pool_media = random.sample(tutti, 15)

# Difficile: prende i 15 con punteggio più alto
pool_difficile = sorted(tutti, key=punteggio, reverse=True)[:15]
```

10. torneo.py — Gestione del Torneo

torneo.py gestisce il bracket a eliminazione diretta con 16 Pokemon. Organizza gli scontri, avanza i vincitori e determina il campione finale.

Struttura del bracket

Il torneo ha 4 round con 15 match totali:

Round	Match	Risultato
Ottavi (Round of 16)	8 match	16 Pokemon → 8 vincitori
Quarti di finale	4 match	8 Pokemon → 4 vincitori
Semifinali	2 match	4 Pokemon → 2 vincitori
Finale	1 match	2 Pokemon → 1 campione

Formato dei dati bracket

bracket_dati è una lista di 4 liste (una per round). Ogni match è un dizionario:

```
match = {
    'a': 'Pikachu', # Nome del primo combattente
    'b': 'Charizard', # Nome del secondo combattente
    'vincitore': None # None = non ancora disputato, altrimenti nome vincitore
}
```

Nel tabellone grafico, ogni box mostra i due nomi del match. Il nome del giocatore appare sempre in viola. Il vincitore ha una freccia ➡ davanti al nome.

11. battaglia.py — Motore di Battaglia

battaglia.py è il cuore della logica di combattimento. Coordina le mosse simultanee, comunica con la GUI tramite le code e gestisce la progressione del torneo.

Flusso di un turno di battaglia

Step	Azione
1	La logica invia 'chiedi_mossa' alla GUI → si attivano i pulsanti
2	Il giocatore clicca una mossa → la GUI invia 'mossa' alla logica
3	La logica chiede alla CPU di scegliere la sua mossa (ai.py)
4	Le due mosse vengono risolte: se uno si cura e l'altro attacca, prima la cura
5	Si invia 'anim_attacco' o 'anim_cura' per le animazioni
6	Si aggiornano gli HP e si invia 'aggiorna' alla GUI
7	Si aggiunge una riga al log con 'log'
8	Se un Pokemon ha HP ≤ 0 , si invia 'anim_ko' e si determina il vincitore
9	La battaglia finisce: si invia 'risultato' e si aspetta 'continua'

Regola delle mosse simultanee

```
# Ordine di risoluzione (priorità):
# 1. Le cure avvengono PRIMA degli attacchi
# 2. Gli attacchi avvengono in ordine di velocità (il più veloce prima)
# 3. In caso di doppio KO: perde il Pokemon più lento

if mossa_gio == 'pozione_normale' or mossa_gio == 'pozione_speciale':
    applica_cura(giocatore) # Prima la cura del giocatore
    if mossa_avv in ['pozione_normale', 'pozione_speciale']:
        applica_cura(avversario) # Poi la cura dell'avversario

# Poi gli attacchi in ordine di velocità
if giocatore.stats['speed'] >= avversario.stats['speed']:
    attacca(giocatore, avversario) # Giocatore attacca prima
    attacca(avversario, giocatore)
else:
    attacca(avversario, giocatore) # Avversario attacca prima
    attacca(giocatore, avversario)
```

12. finestraTk.py — Grafica Pygame

finestraTk.py è il file più grande (~1350 righe). Gestisce tutto ciò che è visibile: rendering, input, animazioni, temi.

Loop principale (avvia)

```
while self.in_esecuzione:
    # 1. Gestione eventi Pygame (click, scroll, tastiera)
    for evento in pygame.event.get(): ...

    # 2. Legge messaggi dalla logica e aggiorna lo stato
    self._leggi_messaggi()

    # 3. Aggiorna posizioni animazioni (scatto, particelle, shake...)
    self._aggiorna_animazioni()

    # 4. Disegna tutto sul buffer
    self._disegna_frame()

    # 5. Mostra il buffer a schermo
    pygame.display.flip()

    # 6. Aspetta per mantenere ~20 FPS
    orologio.tick(1000 // TICK)
```

Metodi di disegno principali

Metodo	Funzione
<code>_txt(x,y,testo,font,col,ancora)</code>	Disegna testo con ancoraggio ('nw','center','e','n'...). Restituisce (w,h).
<code>_rett(x1,y1,x2,y2,...)</code>	Rettangolo semplice con sfondo e/o bordo.
<code>_rett_r(x1,y1,x2,y2,raggio,...)</code>	Rettangolo con angoli arrotondati.
<code>_px(x1,y1,x2,y2,sfondo)</code>	Rettangolo pixel-art con ombra nera offset +3px.
<code>_linea(x1,y1,x2,y2,colore,sp)</code>	Linea retta. sp = spessore.
<code>_cerchio(cx,cy,r,sfondo,bordo)</code>	Cerchio con sfondo e/o bordo.
<code>_barra(x,y,lw,lh,val,max,col)</code>	Barra percentuale colorata (HP, ATK, ecc.).
<code>_overlay()</code>	Sovrappone un rettangolo nero semitrasparente (usato per il risultato).
<code>_btn_continua(label)</code>	Disegna il pulsante Continua/Gioca Ancora centrato in basso.

Gestione delle immagini

Tutte le immagini vengono caricate una sola volta e salvate in cache per non rileggere il disco ad ogni frame.

```
# Cache principale per sprite Pokemon
self.cache_immagini = {} # chiave: (nome, dim, specchiata)

# Cache per immagini di stile (pannelli, sole, luna, nuvole)
self.cache_stile = {} # chiave: (nome_file, larghezza, altezza)
# Nota: cache_stile viene svuotata al cambio tema

# Il nome del file viene derivato dal nome del Pokemon:
# 'Pikachu' → pikachu.png oppure pikachu.png → nomi alternativi con trattini
```

13. Sistema Temi Giorno/Notte

Il gioco ha due temi selezionabili nella schermata difficoltà tramite il toggle in basso al centro. Il cambio tema aggiorna istantaneamente tutti i colori e ricarica le immagini.

Colori dei temi

Variabile	Uso	Tema Scuro	Tema Chiaro
BG	Colore di sfondo principale	#000000 (nero)	#ffffff (bianco)
BG2	Sfondo secondario (zone pannello/log)	#000000	#ffffff
BG3	Sfondo terziario (barre, celle inattive)	#1E1E1E (grigio scuro)	#c8eafc (azzurro chiaro)
ACCENT	Colore principale UI (testi, bottoni)	(255,255,255) bianco	#3D3D3D grigio scuro
TXT	Testo principale	#ffffff bianco	#0c1a2e blu scuro
TXT2	Testo secondario/muted	#a78bba viola chiaro	#1a4a7a blu medio
BORDER	Colore bordi (trasparente in scuro!)	#A3A3A300 (alpha=0)	#000000 nero
GOLD	Oro per la finale	#ffd700	#f57f17
COL_HP	Colore barra HP	#39ff14 verde neon	#2e7d32 verde
BARRA_BG	Sfondo barre HP in battaglia	#000000	#ffffff
BAR_BG	Sfondo barra superiore	#000000	(183,204,209) azzurro

Cosa cambia visivamente con il tema

Schermata difficoltà:

Tema	Aspetto
Scuro	Sfondo nero, stelle animate con scintillio, luna in alto a destra, toggle a destra
Chiaro	Gradiente cielo azzurro (in basso più scuro), nuvole animate, sole in alto a destra, toggle a sinistra

Il BORDER in tema scuro ha alpha=0 (completamente trasparente). Questo significa che le linee dei bracket e le barre HP non hanno bordo visibile, dando un aspetto più minimale. In tema chiaro il bordo è nero.

14. Sistema di Animazioni

Tutte le animazioni sono gestite da `_aggiorna_animazioni()` che viene chiamata ogni frame. Ogni animazione è una struttura dati (dizionario) con `frame/eta` (età) e `durata`.

Animazioni disponibili

Animazione	Descrizione
<code>animazione_scatto</code>	Il Pokemon si lancia verso l'avversario e torna indietro. Dura 10 frame.
<code>animazione_scatto_avv</code>	Come sopra ma per l'avversario (usato nell'attacco doppio).
<code>animazione_ko</code>	Il Pokemon perde opacità gradualmente fino a 0 (dissolvenza). 14 frame.
<code>shake_schermo</code>	L'intera scena trema. Offset X/Y casuali che diminuiscono. 8-10 frame.
<code>scia_attiva</code>	Particelle colorate attorno all'attaccante (attacco speciale). 18 frame.
<code>onde_impatto</code>	Cerchi che si espandono al punto di impatto. 3 cerchi sovrapposti.
<code>particelle_impatto</code>	Scintille che partono dal punto di impatto con gravità. 20 particelle.
<code>speed_lines</code>	Linee orizzontali che indicano il movimento rapido. 8-10 linee.
<code>bolle_cura</code>	Bolle verdi (pozione normale) o viola (speciale) che salgono. 18 bolle.
<code>numeri_fluttuanti</code>	Numeri di danno/cura che salgono e spariscono. Durata 70-100 frame.
<code>nuvole_anim</code>	11 nuvole che si spostano lentamente verso destra. Solo tema chiaro.
<code>stelle_anim</code>	80 stelle con drift lento e scintillio sinusoidale. Solo tema scuro.

Come funziona un'animazione (esempio: scatto)

```

# In _leggi_messaggi, quando arriva 'anim_attacco':
self.animazione_scatto = {'chi': 'giocatore', 'frame': 0, 'durata': 10}

# In _aggiorna_animazioni, ogni frame:
if self.animazione_scatto is not None:
    f = self.animazione_scatto['frame']
    d = self.animazione_scatto['durata']
    p = f / d # progresso da 0 a 1

    # Prima metà: avanza verso l'avversario
    # Seconda metà: torna indietro
    if p < 0.5:
        offset = int(distanza * (p / 0.5))
    else:
        offset = int(distanza * ((1 - p) / 0.5))

    self.offset_x_giocatore = offset
    self.animazione_scatto['frame'] += 1

    if f >= d:
        self.animazione_scatto = None # Animazione finita

```

15. Schermate del Gioco

Schermata Difficoltà

Prima schermata del gioco. Mostra tre pulsanti (Facile, Media, Difficile) e il toggle Giorno/Notte in basso. Sullo sfondo compaiono sprite decorativi di 10 Pokemon ai lati.

Elemento	Dettagli
Pulsanti	Centrati verticalmente, larghezza 420px, altezza 76px ciascuno, gap 18px
Hover	Il pulsante si riempie col colore del bordo quando il mouse passa sopra
Toggle	160×60px, centrato in basso (y = H-110). Click cambia tema e ricarica pannelli
Stelle	80 stelle con posizione casuale (seed=42), drift ±0.15px/frame, scintillio sinusoidale
Nuvole	11 nuvole da cloud1.png a cloud11.png, scalate 1.2x, si spostano da sinistra a destra

Schermata Selezione Pokemon

Permette di sfogliare tutti i 809 Pokemon e sceglierne uno.

Elemento	Dettagli
Pannello sinistro	230×716px. Mostra statistiche del Pokemon selezionato. Sfondo: panel_*.png
Bigpanel	1050×716px. Sfondo della griglia. Immagine: bigpanel_*.png
Griglia	5 colonne × 7 righe visibili. Cella 200×96px, gap 6px. Origine: x=238, y=54
Scrollbar	Verticale, 10px larghezza, in x=W-16. Thumb trascinabile col mouse
Pulsante INDIETRO	Top bar a destra (W-230 → W-120). Torna alla scelta difficoltà
Pulsante INIZIA	Top bar all'estrema destra (W-115 → W-10). Attivo solo se Pokemon selezionato
Difficoltà	Il nome della difficoltà appare centrato nella top bar (verde/arancio/rosso)

Schermata Tabellone

Mostra il bracket del torneo con tutti i match. Il Pokemon del giocatore è evidenziato in viola.

Elemento	Dettagli
Box match	155×56px. Ombra nera +3px. Bordo: viola se contiene il giocatore, verde se ha vincitore
Linee connessione	Collegano i box tra i round. Colore BORDER (trasparente in tema scuro)
Freccia ■	Appare davanti al nome del vincitore in ogni box
Messaggio	Mostra l'esito dell'ultima battaglia (es: 'Hai vinto!') sopra il pulsante Continua

Schermata Battaglia

La schermata principale del combattimento.

Elemento	Dettagli
Wallpaper	Sfondo casuale dalla cartella wallpaper_dark/ o wallpaper_light/. Non si ripete due volte di fila
Sprite giocatore	320×320px, posizione GX=50 GY=80, specchiato orizzontalmente
Sprite avversario	320×320px, posizione AX=W-370 AY=140, non specchiato
Barre HP/DEF/SpD	In alto, larghezza 300px. Mostra valore/massimo a destra
Log	Zona bassa sinistra (72% larghezza). Ultime 14 righe colorate
Pulsanti mossa	Zona bassa destra. 4 pulsanti verticali. Disabilitati durante turno CPU
Overlay risultato	Rettangolo nero semitrasparente + messaggio + Continua al termine del match

16. Aggiungere Contenuti

Nuovi Pokemon

Per aggiungere un Pokemon al gioco:

```
# 1. Aggiungi il Pokemon al file pokedex.json:
{ "nome": "NomePokemon",
  "tipi": ["Tipo1"],
  "stats": {
    "hp": 100, "attack": 80, "defense": 70,
    "sp_attack": 90, "sp_defense": 75, "speed": 85
  }
}

# 2. Aggiungi l'immagine:
# File: PokemonGame/pokemon_images/nomepokemon.png
# Il nome deve essere in minuscolo. Se il nome ha spazi, usa il trattino:
# 'Mr. Mime' → mr.-mime.png
```

Nuovi Wallpaper

Copia file PNG/JPG/WEBP nelle cartelle wallpaper_dark/ o wallpaper_light/. Il gioco li seleziona casualmente ad ogni battaglia e non ripete lo stesso due volte di fila (se ci sono almeno 2 file).

Font personalizzato

```
# Metti i file TTF in: PokemonGame/style/
# PixelifySans-Regular.ttf → testi normali
# PixelifySans-Bold.ttf → titoli e grassetto

# Se i file non esistono, il gioco usa il font bitmap built-in di Pygame
# come fallback automatico (nessun errore).
```

Immagini pannello

File	Dimensione	Uso
panel_scuri.png	230 × 716 px	Sfondo pannello statistiche, tema notte
panel_chiari.png	230 × 716 px	Sfondo pannello statistiche, tema giorno
bigpanel_scuri.png	1050 × 716 px	Sfondo griglia Pokemon, tema notte
bigpanel_chiari.png	1050 × 716 px	Sfondo griglia Pokemon, tema giorno
sun.png	130 × 130 px	Sole nella schermata difficoltà (tema giorno)
moon.png	110 × 110 px	Luna nella schermata difficoltà (tema notte)

cloud1.png ... cloud11.png	qualsiasi	Nuvole animate (scalate 1.2x automaticamente)
----------------------------	-----------	---

Modificare le difficoltà

```
# In ai.py, la funzione che calcola il pool:
def pool_avversari(tutti_pokemon, difficolta):
    punteggio = lambda p: sum(p['stats'].values())
    if difficolta == 'facile':
        return sorted(tutti_pokemon, key=punteggio)[:15] # 15 più deboli
    elif difficolta == 'media':
        return random.sample(tutti_pokemon, 15) # 15 casuali
    else: # difficile
        return sorted(tutti_pokemon, key=punteggio, reverse=True)[:15] # 15 più forti

# Per rendere la modalità facile ancora più semplice,
# abbassa il numero 15 o cambia la formula del punteggio.
```

Appendice — Riferimento Rapido

Shortcut da tastiera

Tasto	Azione
ESC	Alterna tra schermo intero e finestra
Alt+F4	Chiude il gioco (da sistema operativo)

Variabili globali di colore (aggiornate dal tema)

BG	Sfondo principale	TXT2	Testo secondario
BG2	Sfondo secondario	BORDER	Bordi (trasparente in scuro)
BG3	Sfondo terziario	GOLD	Oro per la finale
ACCENT	Colore primario UI	COL_HP	Colore barra HP
ACCENT2	= ACCENT (alias)	COL_DEF	Colore barra DEF (#1565c0)
OK	Verde (vittoria, HP)	COL_SPD	Colore barra SpD
WARN	Arancio (media diff.)	COL_ATK	Colore barra ATK (ff3535)
ERR	Rosso (difficile, danno)	COL_SPA	Colore barra SpA
TXT	Testo principale	COL_VEL	Colore barra VEL

Colori fissi (mai cambiati dal tema)

Costante	Valore	Uso
VIOLA_PLAYER	(168, 85, 247)	Viola per evidenziare il giocatore nel tabellone
COL_DEF	#1565c0	Blu per la barra difesa
COL_SPD	#a855f7	Viola per la barra difesa speciale
COL_ATK	ff3535	Rosso per la barra attacco
COL_SPA	ff9f43	Arancio per la barra attacco speciale
COL_VEL	#e879f9	Rosa per la barra velocità

Fine del Manuale

Questo documento copre ogni aspetto del codice di Pokemon Tournament, dal sistema di comunicazione tra thread alle singole costanti di layout. Per qualsiasi modifica al gioco, il punto di partenza è sempre `finestraTk.py` per la grafica e `battaglia.py` per la logica.

Suggerimento di studio: apri `finestraTk.py` e segui il flusso dalla funzione `avvia()` verso il basso. Ogni schermata ha il suo metodo `_disegna_*`() e ogni animazione viene aggiornata in `_aggiorna_animazioni()`. Le code sono il confine netto tra logica e grafica: tutto ciò che attraversa quel confine è un messaggio dizionario.